

EPICS: Efficient Parallel Pattern Fault Simulation for Sequential Circuits via Strongly Connected Components

Mingjun Wang^{1,2,3}, Hui Wang³, Jianan Mu^{*1,3}, Xinyu Zhang^{1,2}, Bin Sun^{1,2}, Yihan Wen⁴, Zizhen Liu^{1,3}, Feng Gu^{1,2,3}, Jun Gao³, Shengwen Liang^{1,2}, Jing Ye^{*1,2,3}, Xiaowei Li^{1,2}, Huawei Li^{1,2}

¹State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences

²University of Chinese Academy of Sciences ³CASTEST Co., Ltd. ⁴Beijing University of Technology

Abstract—As functional safety of electronic chips gains importance in autonomous vehicles and aerospace, standards like ISO 26262 mandate high diagnostic coverage, requiring extensive gate-level fault simulations. However, for large-scale industrial sequential circuits, these simulations are time-consuming, creating a significant bottleneck in chip development. Prior approaches have focused on reducing computational complexity and optimizing CPU hardware usage by minimizing redundant computations during fault propagation and leveraging bit-level parallel processing capabilities. Techniques like parallel-pattern and event-driven simulations have improved performance in combinational circuits but face limitations in sequential circuits due to timing dependencies within loops. The challenge lies in parallelizing simulations across different cycles without violating these dependencies, which is exacerbated by the complex feedback structures in SCCs. In this work, we propose a novel parallel-pattern fault simulation framework that combines loop fusion with efficient event traversal to accelerate sequential circuit simulations. By compiling simple loops into larger nodes, we reduce the number of feedback events without introducing excessive redundancy. For larger SCCs, we develop specialized algorithms for selecting loop entrance nodes based on indegree analysis and implement the lazy propagation strategy for internal nodes. This approach minimizes simulation events caused by inaccurate predictions and reduces overhead associated with false event propagation. We integrate these techniques into our simulation framework, EPICS, which strategically mixes compiled and event-driven simulations to optimize performance. Experimental results demonstrate that EPICS achieves a 5.94× speedup over state-of-the-art commercial tool while maintaining the same fault coverage.

I. INTRODUCTION

Since sectors like autonomous vehicles and aerospace have become critical to modern life, the functional safety of electronic chips has gained increased attention [1]. ISO 26262 standard [2] outlines safety requirements for automotive-grade chips [3], including MCUs, ASICs, and SoCs [4]. Meeting the requirements demands high diagnostic coverage [5], requiring extensive fault simulations [6]. Fine-grained fault simulation requires injecting faults into gate-level netlist and analyzing output changes under input patterns [7]. Due to the large scale of industrial circuits and numerous fault insertion points, sequential circuit gate-level fault simulation can take weeks [8], creating a significant bottleneck in chip development [9]. Accelerating gate-level fault simulation is urgent [10].

Despite the current apparent and ongoing demand for acceleration, the challenge of speeding up fault simulation in sequential circuits is far from new [11]. This long-standing issue continues to be of critical importance, with significant

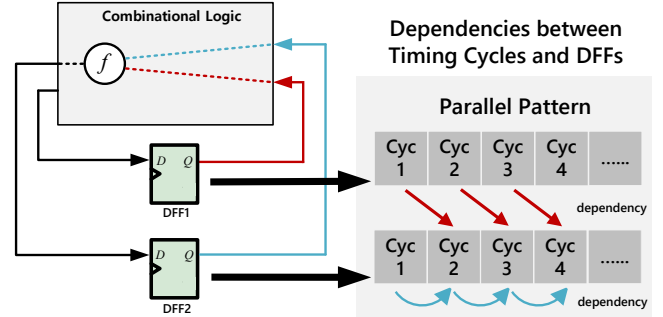


Fig. 1 Feedback Loop Example in Circuit.

room for improvement. Previous research can be divided into two main areas [12]: reducing computational complexity and optimizing for CPU hardware. The focus has been on minimizing redundant computations while propagating good and faulty values across the circuit and efficiently executing bit-level gate simulations on CPUs with high bit-parallel processing capabilities. The concurrent simulation uses event differentials to avoid redundant faults and good values simulations under the same inputs [13]. To better utilize CPUs' multi-bit parallel processing abilities, PARIS introduced the parallel pattern fault simulation [14], and PROOFS introduced the parallel fault simulation [15]. They merge the propagation of multiple single-bit signals into a single word for bit-level parallel processing [16]. In the latest tools, parallel-pattern and event differentials strategies have been used to achieve significant performance improvements in the simulation of combinational circuits [17], [18]. However, for sequential circuits, the use of parallel-pattern or parallel-fault is limited, and further improvement in simulation performance is difficult.

This is because sequential fault simulation with parallel patterns or faults presents challenges. Placing parallel faults on different bits for parallel processing conflicts with concurrent algorithms that consolidate consistent events between fault occurrences. Parallel-pattern simulates inputs from different cycles on different bits. However, there exists a contradiction between parallel patterns and the timing dependencies. As illustrated in Fig. 1, the state of DFF 2 at cycle $n + 1$ depends on the state of DFF 1 at cycle n and the state of DFF 2 at cycle n . This dependency makes it difficult to parallelize the simulation across different cycles. In large-scale sequential circuits, strongly connected components (SCCs), defined in graph theory as nodes that can mutually reach each other

along directed edges, further complicate this issue. The method proposed in [14] predicts the future states of DFFs for 64 cycles and then simulates them. However, due to potential prediction errors, all 64 cycles still need to be executed sequentially to determine the true values of all registers for each cycle. If the actual values of registers for a specific cycle match the predictions, subsequent pathways need not be executed. Nevertheless, updating the predicted future value of a DFF leads to simulations of all cells along the loop. In summary, the timing dependencies introduced by loops limit the performance improvements achievable with parallel patterns in sequential circuit fault simulation.

One insight is to compile and fuse loops into larger nodes, reducing simulation events caused by feedback. However, compiling loops into single nodes can prevent the detection of redundant events within the internal nodes, introducing excessive redundancy. Therefore, we opt to compile only simple loops. When traversing nodes in large SCCs, selecting loop entrance nodes is crucial, as it guides event propagation within the loop. If entrance nodes receive more feedback, prediction errors may increase, leading to greater simulation overhead. Additionally, allowing each node to receive inputs from preceding nodes directly can overly rely on the prediction accuracy of entrance nodes, increasing simulation costs. Therefore, another key insight is developing specialized algorithms for loop entrance selection and node updates in complex, large SCCs to reduce the simulation overhead associated with parallel patterns.

To this end, we propose a parallel-pattern framework that combines loop fusion with efficient event traversal, ultimately surpassing the performance of commercial tools. Initially, we employ a hybrid approach that integrates compiled fusion and event-driven methods within the circuit, merging shorter loop feedbacks within SCCs into single nodes to reduce the number of feedback events. For larger SCCs, we introduce loop entrance selection algorithms based on node indegree analysis and implement node lazy propagation algorithms. These algorithms, in conjunction with a guess-based parallel-pattern mechanism, significantly reduce the number of events propagated on loops caused by inaccurate predictions. We have realized these technologies in a system we call EPICS. Experimental results show that EPICS is faster than the latest commercial tool. Our contributions are summarized as follows:

- We propose a novel traversal framework for parallel-pattern simulation in sequential circuits, specifically designed for SCCs. Our method includes heuristic loop entry point selection based on structural analysis and a lazy propagation strategy for nodes, which reduces the overhead of false event propagation.
- We leverage the strengths of both compiled and event-driven simulations by strategically mixing them within the circuit, optimizing memory access and reducing scheduling overhead during the simulation process.
- Utilizing these techniques, we developed an efficient fault simulation framework for sequential circuits, achieving a 5.94× performance improvement compared to the commercial tool while maintaining the same fault coverage.

II. BACKGROUND AND RELATED WORK

A. Temporal Correlation and Feedback Loops

The complexity of sequential circuits arises from state dependencies and feedback loops [19]. Unlike combinational circuits, the state of a sequential circuit depends on both current inputs and previous states, creating temporal correlations across multiple clock cycles and significantly increasing simulation complexity [20]. Feedback loops introduce cyclic dependencies, represented in graph theory as Strongly Connected Components (SCCs) [21], where any two nodes can reach each other through directed edges [22]. SCCs force simulators to iteratively update circuit states over multiple cycles, adding to the computational burden [23]. Algorithms such as Tarjan's [24] and Kosaraju's [25] are commonly used to identify SCCs, optimizing feedback loop handling and reducing redundant computations [26].

B. Fault Simulation in Sequential Circuits

Fault simulation is essential for verifying functional safety of sequential circuits and evaluating test pattern effectiveness [27], [28]. Stuck-at fault model is commonly used [29]. Due to the need to track state across multiple clock cycles, fault simulation in sequential circuits is inherently more complex than in combinational circuits [30]. Feedback loops further complicate this process, requiring the management of extensive state information and repeated computations [7], [31].

C. Parallel Simulation Techniques

Various parallel simulation techniques have been proposed to alleviate computational bottlenecks [32]. Parallel pattern single fault propagation (PPSFP) simulation method processes multiple input patterns simultaneously, leveraging hardware word length for speedup [14], [33]. While this method excels in combinational circuits [34], state dependencies and feedback loops in sequential circuits limit the achievable parallelism [35], resulting in diminished speedups [36]. Parallel fault simulation reduces redundant computations by simulating multiple faults concurrently [37], yet its efficiency is constrained by fault interdependencies and the overhead of managing large state information in complex circuits [38].

Classic simulators like PROOFS [15] and PARIS [14] employ different optimization strategies. PROOFS uses event-driven parallel fault simulation, focusing on sparse fault effects to minimize unnecessary computation [15], while PARIS is a parallel-pattern fault simulator [14]. Despite their successes in specific scenarios, both struggle with the performance demands of large-scale sequential circuits with intricate feedback loops. Later tools like HOPE [16] and VISION [33] enhance these approaches by optimizing fault dropping and state initialization, improving simulation efficiency.

D. Compiled and Event-Driven Simulation

Sequential circuit simulation typically uses compiled and event-driven methods [39], [40]. Compiled simulation translates circuit logic directly into machine code, eliminating interpretation overhead and improving speed, particularly in

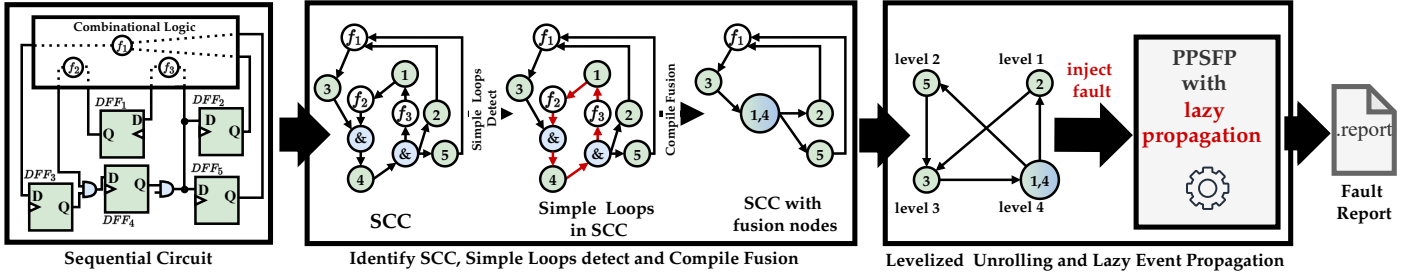


Fig. 2 EPICS Hybrid Fault Simulation Framework.

combinational circuits. However, the exponential growth of state space in sequential circuits due to flip-flops and feedback loops makes the approach less efficient, as it incurs significant compilation and storage overhead. Event-driven simulation triggers computations only when signal changes occur, reducing unnecessary calculations [41]. While effective in circuits with sparse changes, frequent event scheduling in circuits with many feedback loops can result in performance bottlenecks.

III. EPICS FRAMEWORK

Our proposed framework, illustrated in Fig. 2, takes a gate-level circuit, input patterns, and a fault list as inputs to produce a fault simulation report. The framework consists of two main components: Firstly, we process the netlist by identifying the SCCs using Tarjan's algorithm [24]. For simple timing loops within these SCCs, we compile them into single nodes to reduce the number of simulation events. Secondly, for traversing large SCCs, we propose an indegree-based strategy to select the least dependent node as the entry point for speculative SCC traversal. Additionally, we introduce a lazy propagation strategy, where updates are deferred until upstream values confirm that the current node's prediction is incorrect. Finally, the framework outputs a fault coverage report.

IV. COMPILED SIMULATION FOR SIMPLE TIMING LOOPS

In sequential circuit simulation, small timing loops typically consist of a few DFFs and simple combinational logic. Due to their short feedback paths and low dependency, compiled simulation is efficient in handling such loops. Based on SCC identification, we select small timing loops containing 1-2 DFFs and fewer combinational logic gates for compiled simulation. This approach integrates the combinational logic and DFF state transitions into a single functional composite node, avoiding event-driven simulation's frequent event scheduling overhead. By processing machine word length bit pattern data in parallel, the system simulates multiple states in a single event until the circuit reaches a stable state. Compiled simulation consolidates state changes over multiple clock cycles into one computation, significantly enhancing simulation efficiency.

As shown in Fig. 3, in the feedback loop, input a and b affect the states of DFFs A and B through logic gates C and D, and these state changes are fed back into the logic gates, forming a closed loop. Compiled simulation converts this loop into an equivalent instruction sequence, eliminating the need for frequent scheduling in event-driven simulations. State updates

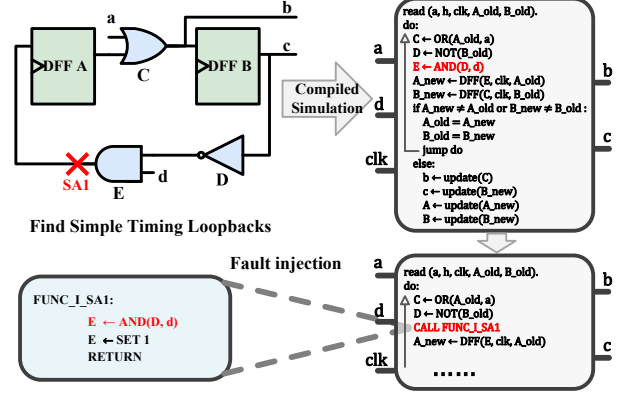


Fig. 3 Compiled Simulation and Fault Injection Example

and output computations for each clock cycle are handled in parallel until the feedback loop reaches a stable state.

Meanwhile, fault injection is achieved by modifying the compiled instruction sequence. For example, Fig. 3 shows a stuck-at-1 (SA1) fault injected at gate E, where the output of E is forced to 1, propagating the fault through the circuit, reducing memory access delays and improving fault simulation efficiency. Fault injection directly modifies the instruction sequence with minimal changes to the original instructions.

Compiled simulation is particularly well-suited for efficiently simulating small timing loops. Combined with fault injection techniques, this significantly reduces simulation time and avoids the cyclic scheduling issues of event-driven simulations while maintaining accuracy and improving performance.

V. LEVELIZED UNROLLING AND LAZY EVENT PROPAGATION

The loop simulation requires starting from an entry point, traversing the loop, and feeding the timing values back to the entry. Since the entry node guides the value updates for all nodes in the loop, the accuracy of the entry node's predicted value significantly impacts the number of simulations required. Therefore, we select the node with the highest prediction accuracy as the loop entry. Upon establishing the foundation of each loop, we propose a lazy propagation strategy at each node. To prevent significant redundant computations across all nodes on a loop due to inaccurate predictions at the entrance, each node refrains from readily accepting event propagation from higher levels until it can ascertain the presence of errors in subsequent predictions.

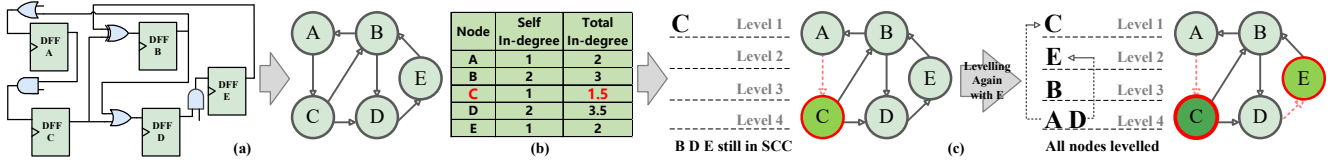


Fig. 4 Indegree-Based Levelized Unrolling of SCC.

A. Indegree-based Levelized Unrolling of SCC

Given the complexity of the circuit structure and connections, we propose a heuristic method based on analyzing the structure of the DFF graph. Through experimental analysis, we found that if a DFF depends on more sequential nodes—i.e., it has a higher indegree—its future cycle values become more uncertain and harder to predict. Conversely, a DFF with a lower indegree is more likely to generate future values based on its own past state, resulting in lower uncertainty.

Based on this observation, we propose Algorithm 1, which uses indegree analysis to identify efficient entry points for SCC simulation. Algorithm 1 consists of three main steps: calculating indegrees, selecting entry points, and then leveling the nodes based on the selected entry points. We illustrate this process using an example shown in Fig. 4.

Algorithm 1: Indegree-based Levelized Unrolling of Nodes in SCC

Input: SCC of a circuit graph
Output: Hierarchically unrolled node sequence L

```

1 Layer list  $L \leftarrow \emptyset$ , Processed node set  $U \leftarrow \emptyset$ ;
2 Current layer  $level \leftarrow 0$ ;
3 foreach  $v \in SCC$  do
4    $d_{self}(v) \leftarrow$  self indegree;
5    $d_{pred}(v) \leftarrow$  predecessor indegree;
6    $d_{total}(v) \leftarrow d_{self}(v) + \frac{1}{2}d_{pred}(v)$ ;
7 while  $SCC \neq \emptyset$  do
8    $v_{min} \leftarrow \arg \min_{v \in SCC} d_{total}(v)$ ;
9   Cut input edges of  $v_{min}$ ;
10  if  $L[level]$  does not exist then
11     $L[level] \leftarrow \emptyset$ ;
12     $L[level] \leftarrow L[level] \cup \{v_{min} \cup$ 
13      nodes with all predecessors in higher layers};
14     $SCC \leftarrow SCC \setminus L[level]$ ;
15     $U \leftarrow U \cup L[level]$ ;
16    if cycles remain in  $SCC$  then
17      continue;
18     $level \leftarrow level + 1$ ;
19 return  $L$ ;
```

The first step (lines 4-5 of the algorithm) calculates the self indegree ($d_{self}(v)$) and predecessor indegree ($d_{pred}(v)$) for each node v in the SCC, and then computes the total indegree ($d_{total}(v)$) using the following formula:

$$d_{total}(v) = d_{self}(v) + \frac{1}{2} \sum_{u \in \text{Predecessors}(v)} d_{self}(u). \quad (1)$$

In Fig. 4(b), each node's calculated indegrees are shown. For instance, node C has a self-indegree of 1 and a total indegree of 1.5, making it a suitable candidate for the initial levelized unrolling step due to its lower dependency on other nodes.

Next, lines 7-10 of the algorithm select the node with the smallest total indegree— v_{min} —and begin the unrolling process. As shown in Fig. 4(c), node C is selected first because it has

the smallest total indegree. This node is then placed in Level 1 of the unrolling hierarchy, and its input edges are temporarily disconnected to break any cyclical dependencies.

The algorithm then places v_{min} and all other nodes whose predecessors are already assigned to higher layers into the current layer. This ensures that nodes dependent on already processed nodes are handled in a topologically valid unrolling.

If cycles still exist in the SCC after the initial unrolling (lines 13-14 of the algorithm), the process is repeated by selecting the next node with the smallest total indegree. In Fig. 4(c), nodes B, D, and E still form an SCC after processing C. The algorithm proceeds by selecting node E due to its lower total indegree and continues the unrolling process.

Finally, the current layer is incremented, and the algorithm proceeds until all nodes are unrolled, as illustrated in Fig. 4(c), where all nodes are assigned to respective levels. The result is a hierarchical node sequence that can be efficiently simulated while minimizing redundant computations.

Algorithm 2: Lazy Event Propagation

Input: Sequential circuit with DFFs, SCCs, initial states, and input stimuli
Output: Updated circuit states and simulation results

```

1 Initialize a lazy event table  $L$  to record lazy events;
2 Set  $current\_time\_point = 0$ ;
3 while Simulation is not complete do
4   for each DFF  $v$  do
5     if Event triggered at  $current\_time\_point$  then
6       Update state of  $v$ ;
7       Propagate event to dependent nodes;
8       Clear lazy events of  $v$  from  $L$ ;
9     if Lazy event exists for  $v$  at  $current\_time\_point$  in  $L$  then
10      Propagate event to dependent nodes;
11      Clear lazy events of  $v$  from  $L$ ;
12   for each DFF  $v$  do
13     if  $v$  is scheduled to change at a future time  $t'$  then
14       Update state of  $v$ ;
15       Record lazy event for  $v$  at time  $t'$  in  $L$ ;
16   if All events at  $current\_time\_point$  are processed then
17     Increment  $current\_time\_point$ ;
18 return Updated circuit states and simulation results;
```

B. Lazy Event Propagation

Building on the hierarchical unfolding of SCCs, we propose a lazy propagation algorithm 2 aimed at reducing unnecessary state propagation and event scheduling. This strategy is particularly effective when managing complex SCCs and fault propagation. The core idea is to propagate events only when absolutely necessary, thus avoiding redundant computations.

The first part of algorithm 2 (lines 1-2) initializes a lazy event table, which records events that are postponed to be processed at an appropriate time. This event table helps the

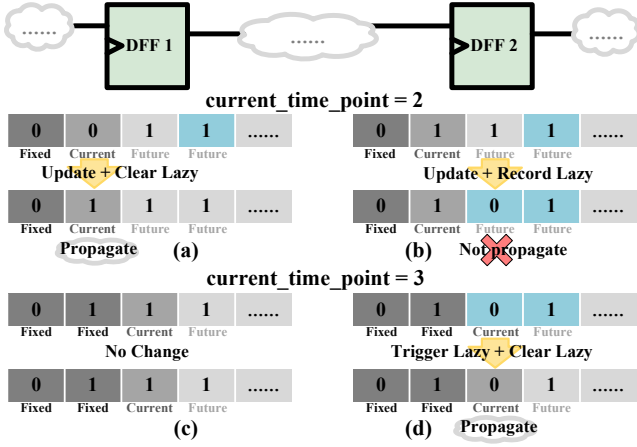


Fig. 5 Lazy event propagation in sequential circuits (lazy events are indicated in blue).

simulator efficiently manage which events need immediate processing and which can be delayed.

Figure 5(a) illustrates a scenario where DFF1 has an event triggered at `current_time_point = 2`. At this moment, DFF1's state is updated, and the event is propagated to dependent nodes. Simultaneously, DFF1's lazy event record is cleared to ensure no redundant computations occur. This behavior is described in lines 5-8 of the algorithm. If no event is triggered at a clock cycle (such as `current_time_point = 3` in Figure 5(c)), the algorithm simply waits without making any state changes, thereby reducing unnecessary propagations and scheduling.

Lines 13-15 in algorithm 2 handle situations where state changes are scheduled to occur in the future. In Figure 5(b), at `current_time_point = 2`, DFF2 records a lazy event to be triggered at `current_time_point = 3` and updates itself, but the event is not propagated immediately. This delayed processing helps avoid multiple unnecessary event propagates. When the future clock cycle arrives (as in `current_time_point = 3` in Figure 5(d)), the lazy event is processed, DFF2's event is propagated forward, and DFF2's lazy event record is cleared. This process is described in lines 10-11, ensuring unnecessary propagations do not occur before the appropriate time.

Finally, once all events for a clock cycle have been processed, `current_time_point` increments, as described in line 17 of algorithm 2, and the simulator continues to process pending events for the next clock cycle. This design allows the simulator to perform simulations efficiently, minimizing computation while ensuring correctness.

VI. EXPERIMENTAL RESULTS

A. Experimental Settings

We implemented EPICS in C++ and evaluated it on a Red Hat Enterprise Linux Server 7.9, equipped with an Intel Xeon® Platinum 8260 CPU @ 2.40GHz and 1.5TiB of RAM. For our evaluation, we compared EPICS against two other simulators that support gate-level fault simulation, representing both commercial and academic tools. The first is Z01X [42], a state-of-the-art commercial simulator. The second is VISION [33], a well-known academic simulator. Since the results

TABLE I Statistics of Benchmark Circuits

Circuit	#Gates	#Cycles
b20	19,358	20,000
b22	28,929	20,000
b18	103,838	20,000
openC906	624,058	20,000
openC910	5,323,400	20,000
Comm1	78,739	28,286
Comm2	3,103,038	36,800
Comm3	4,300,720	23,480

reported for VISION in [33] were obtained on an outdated Sun Sparc 2 workstation, we reimplemented VISION to run on our modern hardware platform, resulting in a version we refer to as VISION+. This reimplemented version demonstrated significantly shorter simulation times than the original, owing to modern hardware advancements and optimizations. To ensure a fair and consistent comparison, we evaluated all simulators, including EPICS, Z01X, and VISION+, on the same platform and under the same settings.

Benchmark Circuits. We evaluated EPICS on a diverse set of benchmark circuits, including both academic and industrial designs, to assess its scalability and applicability. TABLE I summarizes the benchmarks circuits used in this paper, which cover small cores in academic papers ITC'99 [43] (*b20*, *b22*, *b18*), open-source RISC-V CPUs [44] (*openC906*, *openC910*), and commercial processors (*Comm1*, *Comm2*, *Comm3*). In our experiments, real functional patterns were used for commercial processors (*Comm1*, *Comm2*, *Comm3*), while random patterns were applied to other circuits. The simulation cycles for each circuit are also shown in TABLE I.

Fault list and fault coverage. We generated stuck-at faults for gates and DFFs in the circuits and set observation points at all outputs. The performance of each simulation was evaluated and the simulation runtime was reported. The fault coverage achieved by our proposed framework is consistent with that of the commercial tool Z01X across all benchmarks, demonstrating the correctness of the proposed fault simulation framework.

B. Evaluation on Benchmarks and Comparisons

We conducted performance tests on all the aforementioned benchmark circuits and compared the runtime of the proposed EPICS with that of the baseline methods. The experimental results are shown in Fig. 6, including the runtime of each algorithm and the corresponding speedup ratios. As observed, the proposed EPICS achieves the best performance across all benchmark circuits. For a small circuit *b20* with 19,358 gates, the runtime of Z01X, VISION+, and EPICS are 405s, 172s, and 42s, respectively. For the commercial circuit *Comm3* with 4,300,720 gates, the runtime of Z01X, VISION+, and EPICS are 82,968s, 117,314s, and 22,084s, respectively. For the largest circuit *openC910* with 5,323,400 gates, the runtime of Z01X, VISION+, and EPICS are 26,904s, 23,906s, and 6,719s, respectively. On average, EPICS achieves a $5.94\times$ speedup over Z01X, and a $4.23\times$ speedup over VISION+ across all benchmark circuits. The experimental results demonstrate that the proposed EPICS exhibits excellent simulation performance

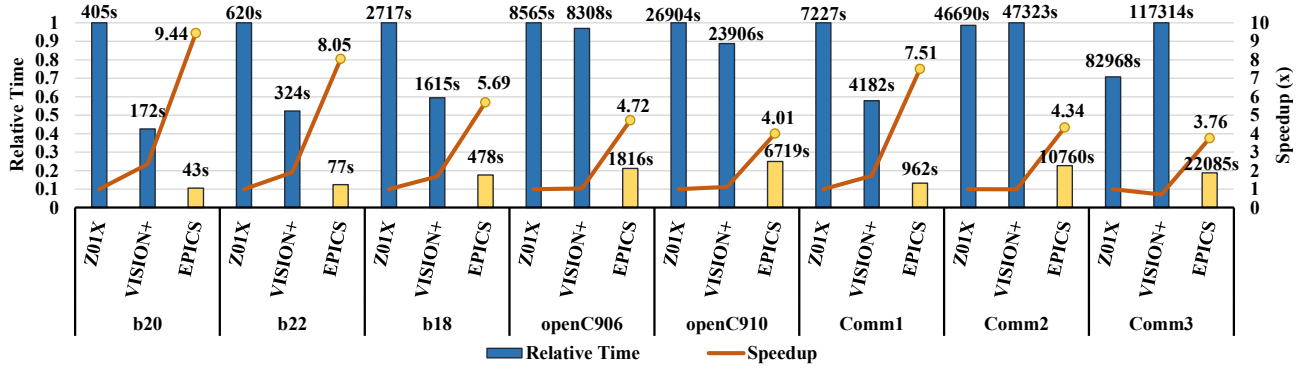


Fig. 6 Performance comparison of various simulators.

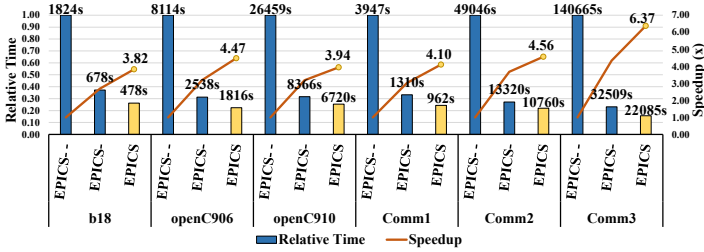


Fig. 7 Ablation Study on Optimizations: EPICS-- (No Optimization), EPICS- (Includes the SCC traversal strategy), EPICS.

across circuits of various scales, achieving significant acceleration compared to both the commercial tool and the state-of-the-art academic approach. Notably, EPICS reduces the simulation time for the commercial circuit Comm3 from approximately 22 hours with commercial tools to just 6 hours.

C. Ablation study

We conducted an ablation study to evaluate the acceleration effect of small loop compilation and SCC traversal strategy. We implemented another two versions of EPICS: EPICS--, which excludes both the small loop compilation and SCC traversal strategies, and EPICS-, which includes the SCC traversal strategy but excludes small loop compilation. Aside from these optimizations, both versions have the same implementation as the original EPICS. The results are listed in Fig. 7. It shows that EPICS- achieves 3.28 $\times$ speedup over EPICS-- on average. Meanwhile, EPICS achieves 1.32 $\times$ speedup over EPICS- on average, and 4.33 $\times$ speedup over EPICS--. These indicate that both the SCC traversal strategy and the compilation of small loops contribute significantly to simulation acceleration.

TABLE II compares the total number of simulation events processed by each version. EPICS demonstrates a significant reduction in event count, which directly contributes to its reduced simulation time overhead. The compilation for small SCCs reduces event-driven overhead by eliminating the need for frequent event scheduling within these regions. As a result, EPICS processes 51% fewer events than EPICS-- in b18. The SCC traversal strategy is crucial in minimizing event count, especially in circuits with complex feedback loops. In Comm3, EPICS processes only 13.6% of the events required

TABLE II Event Counts and Ratios Compared to EPICS

Circuit	EPICS Events	EPICS- Events	EPICS- Ratio($\times$)	EPICS-- Events	EPICS-- Ratio($\times$)
b20	1.06e9	1.31e9	1.24	3.75e9	3.55
b22	1.71e9	2.29e9	1.34	7.21e9	4.21
b18	1.31e10	1.98e10	1.51	5.51e10	4.20
openC906	1.69e10	2.48e10	1.47	8.02e10	4.74
openC910	6.13e10	9.49e10	1.55	2.60e11	4.24
Comm1	1.04e10	1.59e10	1.52	4.43e10	4.24
Comm2	1.10e11	1.62e11	1.48	4.95e11	4.50
Comm3	2.38e11	3.97e11	1.67	1.75e12	7.35
Average	-	-	1.47	-	4.63

Events: Number of simulation events generated.

Ratio ($\times$): Event count ratio compared to EPICS.

by EPICS--, demonstrating the efficiency of these combined strategies. On average, EPICS achieves a 47% reduction in event count compared to EPICS- and a 4.63 $\times$ reduction compared to EPICS--, confirming the impact of its optimizations.

The performance gains achieved by EPICS stem from the synergistic effect of its optimizations. Compiled simulation reduces event scheduling overhead in simple loops, while the SCC traversal strategy avoids unnecessary event propagation. Together, these optimizations enable EPICS to efficiently handle SCCs with minimal computational overhead.

VII. CONCLUSION

This paper introduced EPICS, an efficient framework for parallel-pattern fault simulation in sequential circuits, addressing the challenges of strongly connected components (SCCs). Combining compiled simulation, lazy event propagation, and hierarchical SCC unrolling, EPICS reduces computational overhead and enhances performance without sacrificing fault coverage. Experimental results show that EPICS achieves 5.94 $\times$ speedup over commercial tool Z01X while maintaining equivalent fault coverage, demonstrating its scalability for large-scale industrial designs.

ACKNOWLEDGEMENT

This paper is supported in part by the National Natural Science Foundation of China (NSFC) under grant No. (92373206, 92473203). The corresponding authors are Jianan Mu and Jing Ye.

REFERENCES

- [1] F. J. Belmonte, S. Martín, E. Sancristobal, J. A. Ruipérez-Valiente, and M. Castro, "Overview of embedded systems to build reliable and safe adas and ad systems," *IEEE Intelligent Transportation Systems Magazine*, vol. 13, no. 4, pp. 239–250, 2020.
- [2] G. Griessnig and A. Schnellbach, "Development of the 2nd edition of the iso 26262," in *Systems, Software and Services Process Improvement: 24th European Conference, EuroSPI 2017, Ostrava, Czech Republic, September 6–8, 2017, Proceedings 24*. Springer, 2017, pp. 535–546.
- [3] W. Chen and J. Bhadra, "Practices and challenges for achieving functional safety of modern automotive socs," *IEEE Design & Test*, vol. 36, no. 4, pp. 31–47, 2019.
- [4] K. Suo, L. Vu, M. R. Islam, N. Dhar, T. N. Nguyen, S. He, and X. Wu, "A systematic investigation of hardware and software in electric vehicular platform," in *Proceedings of the 2024 ACM Southeast Conference*, 2024, pp. 9–17.
- [5] W. Granig, D. Hammerschmidt, and H. Zangl, "Calculation of failure detection probability on safety mechanisms of correlated sensor signals according to iso 26262," *SAE International Journal of Passenger Cars-Electronic and Electrical Systems*, vol. 10, no. 2017-01-0015, pp. 144–155, 2017.
- [6] L.-T. Wang, C.-W. Wu, and X. Wen, *VLSI test principles and architectures: design for testability*. Elsevier, 2006.
- [7] A. Miczo, *Digital logic testing and simulation*. John Wiley & Sons, 2003.
- [8] M. M. Peng and J. S. Kuang, "A gpu-based fault simulator for small-delay faults," *Advanced Materials Research*, vol. 753, pp. 2235–2242, 2013.
- [9] R. Baranowski, S. Di Carlo, N. Hatami, M. E. Imhof, M. A. Koche, P. Prinetto, H.-J. Wunderlich, and C. G. Zoellin, "Efficient multi-level fault simulation of hw/sw systems for structural faults," *Science China Information Sciences*, vol. 54, pp. 1784–1796, 2011.
- [10] M. Wang, H. Wang, J. Mu, Z. Liu, J. Gao, J. Ye, H. Li, and X. Li, "Efficient functional safety method for gate-level fine-grained digital circuits with iso-26262," in *2024 IEEE International Test Conference in Asia (ITC-Asia)*. IEEE, 2024, pp. 1–6.
- [11] R. B. Mueller-Thuns, D. G. Saab, R. F. Damiano, and J. A. Abraham, "Vlsi logic and fault simulation on general-purpose parallel computers," *IEEE transactions on computer-aided design of integrated circuits and systems*, vol. 12, no. 3, pp. 446–460, 1993.
- [12] M. Maddela, "Elec 7250 sequential parallel fault simulator."
- [13] M. Abramovici, M. A. Breuer, and K. Kumar, "Concurrent fault simulation and functional level modeling," in *Proceedings of the 14th Design Automation Conference*, 1977, pp. 128–137.
- [14] N. Gouders and R. Kaibel, "Paris: A parallel pattern fault simulator for synchronous sequential circuits," in *1991 IEEE International Conference on Computer-Aided Design Digest of Technical Papers*. IEEE, 1991, pp. 542–545.
- [15] T. M. Niermann, W.-T. Cheng, and J. H. Patel, "Proofs: A fast, memory-efficient sequential circuit fault simulator," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 11, no. 2, pp. 198–207, 1992.
- [16] H. K. Lee and D. S. Ha, "Hope: An efficient parallel fault simulator for synchronous sequential circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 15, no. 9, pp. 1048–1058, 1996.
- [17] Synopsys, *TestMAX ATPG and TestMAX Diagnosis User Guide*, Synopsys, July 2021.
- [18] Mentor, *Tessent® Scan and ATPG User's Manual*, document revision 18 ed., Mentor, A Siemens Business, 2020.
- [19] K.-T. Cheng and V. D. Agrawal, "A partial scan method for sequential circuits with feedback," *IEEE Transactions on Computers*, vol. 39, no. 4, pp. 544–548, 1990.
- [20] N. Miskov-Zivanov and D. Marculescu, "Multiple transient faults in combinational and sequential circuits: A systematic approach," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 29, no. 10, pp. 1614–1627, 2010.
- [21] J. F. Hurford and T. C. Hartrum, "Improving conservative vhdl simulation performance by reduction of feedback," *ACM SIGSIM Simulation Digest*, vol. 26, no. 1, pp. 196–201, 1996.
- [22] H. He, T. Xu, J. Chen, Y. Cui, and J. Song, "A granulation strategy-based algorithm for computing strongly connected components in parallel," *Mathematics*, vol. 12, no. 11, p. 1723, 2024.
- [23] D. V. Overhauser, *Fast timing simulation of MOS VLSI circuits*. University of Illinois at Urbana-Champaign, 1989.
- [24] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM journal on computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [25] J. E. Hopcroft, J. D. Ullman, and A. V. Aho, *Data structures and algorithms*. Addison-wesley Boston, MA, USA., 1983, vol. 175.
- [26] C. A. Larsen, S. M. Schmidt, J. Steensgaard, A. B. Jakobsen, J. v. de Pol, and A. Pavlogiannis, "A truly symbolic linear-time algorithm for sec decomposition," in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2023, pp. 353–371.
- [27] A. H. Khan, Z. H. Khan, and Z. Weiguo, "Model-based verification and validation of safety-critical embedded real-time systems: Formation and tools," in *Embedded and Real Time System Development: A Software Engineering Perspective: Concepts, Methods and Principles*. Springer, 2013, pp. 153–183.
- [28] T. Ahmed et al., "Evaluation of fibonacci test pattern generator for cost effective testing," 2013.
- [29] S. Mourad and E. J. McCluskey, "Fault models," in *Testing and Diagnosis of VLSI and ULSI*. Springer, 1988, pp. 49–68.
- [30] K. Cho and R. E. Bryant, "Test pattern generation for sequential mos circuits by symbolic fault simulation," in *Proceedings of the 26th ACM/IEEE Design Automation Conference*, 1989, pp. 418–423.
- [31] M. Mojtahedi and W. Geisselhardt, "New methods for parallel pattern fast fault simulation for synchronous sequential circuits," in *Proceedings of 1993 International Conference on Computer Aided Design (ICCAD)*. IEEE, 1993, pp. 2–5.
- [32] H.-P. Chang, S. G. Chappell, C. H. Elmendorf, and L. D. Schmidt, "Comparison of parallel and deductive fault simulation methods," *IEEE Transactions on Computers*, vol. 100, no. 11, pp. 1132–1138, 1974.
- [33] R. Nair and D. S. Ha, "Vision: An efficient parallel pattern fault simulator for synchronous sequential circuits," in *Proceedings 13th IEEE VLSI Test Symposium*. IEEE, 1995, pp. 221–226.
- [34] D. Harel, R. Sheng, and J. Udell, "Efficient single fault propagation in combinational circuits," *Communications of the ACM*, vol. 29, no. 4, pp. 300–311, 1986.
- [35] C.-P. Kung and C.-S. Lin, "Parallel sequence fault simulation for synchronous sequential circuits," *Journal of Electronic Testing*, vol. 9, pp. 267–277, 1996.
- [36] J. Kōusaar, R. Ubar, S. Kostin, S. Devadze, and J. Raik, "Parallel critical path tracing fault simulation in sequential circuits," in *2018 25th International Conference "Mixed Design of Integrated Circuits and System" (MIXDES)*. IEEE, 2018, pp. 305–310.
- [37] E. Rudmocl and J. H. Patel, "Overcoming the serial logic simulation bottleneck in parallel fault simulation," in *Proceedings Tenth International Conference on VLSI Design*. IEEE, 1997, pp. 495–501.
- [38] E. M. Rudnick, T. M. Niermann, and J. H. Patel, "Methods for reducing events in sequential circuit fault simulation," in *ICCAD*. Citeseer, 1991, pp. 546–549.
- [39] M. Kooli and G. Di Natale, "A survey on simulation-based fault injection tools for complex systems," in *2014 9th IEEE International Conference on Design & Technology of Integrated Systems in Nanoscale Era (DTIS)*. IEEE, 2014, pp. 1–6.
- [40] B. L. Keller, D. P. Carlson, and W. Maloney, "The compiled logic simulator," *IEEE Design & Test of Computers*, vol. 8, no. 1, pp. 21–34, 1991.
- [41] B. Becker, M. Keim, and R. Krieger, "Hybrid fault simulation for synchronous sequential circuits," *Journal of Electronic Testing*, vol. 15, pp. 219–238, 1999.
- [42] Synopsys, Inc., *Z01X™ Simulator Manufacturing Assurance User Guide*, Synopsys, Inc., Sep. 2023, copyright © 2023 Synopsys, Inc.
- [43] F. Corno, M. S. Reorda, and G. Squillero, "Rt-level itc'99 benchmarks and first atpg results," *IEEE Design & Test of computers*, vol. 17, no. 3, pp. 44–53, 2000.
- [44] "OpenXuantie," 2021, [Online]. Available: <https://github.com/T-head-Semi/>.